

Documentación Universitaria del Proyecto de Lenguaje de Programación: PyScript

Asignatura: Lenguajes y Autómatas II

Institución: Instituto Tecnológico de Nuevo León

Fecha: 23 de septiembre de 2026

1. NOMBRE DEL PROYECTO

PyScript — Compilador académico de lenguaje tipado estático con destino a JavaScript y Entorno Visual Interactivo.

2. OBJETIVO DEL PROYECTO

Diseñar e implementar un compilador completo para un lenguaje de programación con tipado estático y explícito que compile hacia JavaScript, abarcando formalmente las fases de Análisis Léxico, Análisis Sintáctico (generación de Árbol de Sintaxis Abstracta / AST) y Análisis Semántico con gestión de tabla de símbolos.

3. OBJETIVO SECUNDARIO DEL PROYECTO

Desarrollar una plataforma Playground web interactiva integrada que permita la depuración paso a paso (fase por fase), la inspección visual dinámica del AST mediante grafos jerárquicos (D3.js) y la visualización en tiempo real de la tabla de símbolos y diagnósticos del compilador.

4. JUSTIFICACIÓN DEL PROYECTO

El estudio de los compiladores es crucial en la formación de la Ingeniería en Sistemas; sin embargo, comprender el comportamiento interno de algoritmos de parsing, árboles de sintaxis abstracta y la resolución de ámbitos semánticos suele ser abstracto. PyScript resuelve este problema combinando la construcción rigurosa de las fases front-end del compilador (TypeScript estricto) con una herramienta gráfica interactiva moderna (React/Tailwind/D3)

que expone la representación interna del programa durante el proceso de compilación.

5. VENTA DEL PROYECTO

PyScript no es solo un compilador experimental; es un kit pedagógico interactivo diseñado para instituciones educativas e investigadores que buscan enseñar la teoría de autómatas, gramáticas formales y análisis semántico de manera visual, reduciendo la curva de aprendizaje mediante herramientas de inspección directa del AST y un depurador paso a paso estilo Jupyter.

6. REQUERIMIENTOS DEL PROYECTO

Requerimientos Humanos

- 1 Desarrollador / Estudiante de Ingeniería con conocimientos en teoría de lenguajes, gramáticas formales EBNF, estructuras de datos y desarrollo web en TypeScript/React.

Requerimientos Financieros

- **Costo Directo:** \$0.00 USD (Proyecto desarrollado enteramente con herramientas, librerías y tecnologías de código abierto/Open Source).

Requerimientos Tecnológicos

Categoría	Especificación
Lenguaje de implementación	TypeScript (ES2020) en modo estricto
Plataforma web	Node.js (v18+), Vite 8, React 19, Tailwind CSS 4
Visualización	D3.js v7
Formato de Gramática	Notación EBNF (Extended Backus-Naur Form)

7. VENTAJAS DEL PROYECTO

- **Seguridad de Tipos:** Detección de errores semánticos en tiempo de compilación mediante tipado explícito.
- **Visualización Intuitiva:** Exploración gráfica del AST en árbol de nodos interactivo y tabla dinámica de símbolos.
- **Depuración Didáctica:** Motor de ejecución paso a paso (Step Debugger) que resalta la coincidencia de líneas de código fuente con los snapshots del compilador.
- **Generación de Salida Estándar:** Destino de compilación directo hacia JavaScript, garantizando ejecución web o en runtime Node.js.

8. DESVENTAJAS

- **Inexistencia de funciones de usuario:** No se soporta la declaración de funciones personalizadas por el desarrollador en la v0.1.
- **Manejo restringido de estructuras:** No se permiten arreglos multidimensionales (`a[x][y]`) ni estructuras de datos complejas personalizadas.
- **Sintaxis de ciclos acotada:** La cabecera del ciclo `for` solo admite inicializaciones primitivas simples.

9. LIMITANTES

- **Alcance restringido (v0.1):** Exclusión intencional de características como clases, herencia, módulos, excepciones, inferencia de tipos y genéricos definidos por el usuario.
- **Constantes restringidas:** El modificador `const` está limitado únicamente a tipos primitivos (`int`, `float`, `bool`, `string`) y requiere valor inicial obligatorio.
- **Acceso a colecciones:** El acceso por índice en arreglos está limitado a un único nivel (`identificador[expresion]`).

10. RESTRICCIONES

1. **Estructura obligatoria del programa:** Todo código fuente debe iniciar con `program <Identificador> {` y finalizar con `}`.

2. **Punto y coma obligatorio:** Todas las declaraciones, asignaciones y sentencias de E/S deben terminar formalmente con `;`. Los bloques de código `{ }` no llevan punto y coma final.
3. **Condiciones estrictamente booleanas:** Las expresiones evaluadas en estructuras condicionales `if` y de control `for` deben resolver indefectiblemente a tipo `bool`.
4. **Inmutabilidad de constantes:** Prohibida la reasignación o lectura directa de entrada (`read`) sobre símbolos declarados como `const`.

11. MEJORAS

- **Generación de Código Intermedio (IR):** Implementar la representación de código de tres direcciones (TAC).
- **Optimizador de Código:** Incorporar pasos de optimización sobre el IR como Constant Folding (plegado de constantes) y Dead Code Elimination (eliminación de código inalcanzable).
- **Generador de Código JS y CLI:** Construir el pipeline final generador de código objeto JavaScript funcional y un CLI para compilación desde terminal (`pyscript main.pys -o main.js`).

12. RECOMENDACIONES

- Utilizar nombres descriptivos en las variables y constantes para mantener coherencia en las inspecciones semánticas de la tabla de símbolos.
- Mantener la modularidad en la estructura del compilador separando strictly los archivos `lexer.ts`, `parser.ts`, `ast.ts`, `symbolTable.ts` y `semanticAnalyzer.ts`.
- Probar los programas fuente a través del modo paso a paso en el Playground para validar la construcción modular del AST antes de realizar la revisión semántica.

13. OBSERVACIONES

Análisis Técnico del Compilador PyScript (Léxico, Sintáctico y Semántico)

A continuación se detalla cómo se integraron los elementos requeridos dentro de las fases del compilador PyScript:

A. Instrucciones de Entrada, Salida, Condición y Cíclicas

- **Entrada (`read`):** Recibe como argumento un destino válido (`target`, p. ej., variable o `arreglo[indice]`). En el analizador semántico, se verifica que la variable esté declarada y que no sea constante (`mutable: true`).
- **Salida (`print`):** Evalúa cualquier expresión válida (literales, operaciones aritméticas o retornos como `stack.pop()`) e imprime el resultado.
- **Condicionales (`if / else`):** Permite bifurcaciones lógicas. El analizador semántico valida que la condición sea estrictamente de tipo `bool` y crea ámbitos independientes (`enterScope / exitScope`) tanto para la rama `then` como para la rama `else` (incluyendo `else if`).
- **Cíclicas (`for` y `while`):** El ciclo `for` consta de tres partes (inicialización, condición y actualización). Durante la visita semántica (`visitForStatement / visitWhileStatement`), se abre un ámbito local y se valida que la condición central sea una expresión booleana.

B. Constantes, Variables, Arreglos y Secuencia de Block

- **Variables:** Declaradas explícitamente con o sin valor inicial: `int x = 10;` o `float price;`. Se registran en la Tabla de Símbolos como `kind: 'variable'` y `mutable: true`.
- **Constantes:** Declaradas obligatoriamente con el modificador `const` y un valor inicial: `const int MAX = 100;`. Se registran como `kind: 'constant'` y `mutable: false`.
- **Arreglos:** Tipados primitivamente (`T[]`), como `int[] numbers = [10, 20, 30];`. El analizador semántico exige que los literales de arreglo contengan elementos homogéneos y que las expresiones de índice (`a[index]`) sean de tipo entero (`int`).

- **Estructuras Tipadas Adicionales:** Stacks (`stack<T>`) y Queues (`queue<T>`), con métodos integrados (`push`, `pop`, `enqueue`, `dequeue`, `isEmpty`, `size`) validados mediante `visitMethodCall`.
- **Secuencia de Block:** Un bloque delimita un conjunto de instrucciones entre llaves `{ statementList }`. Cada bloque activa un nivel en la pila de scopes de la Tabla de Símbolos (`Map[ ]`), garantizando que las variables locales dejen de existir al salir del bloque (Scope Isolation) y resolviendo el ocultamiento de variables (Shadowing) mediante recorridos inversos.

C. Operadores Aritméticos, Relacionales y Lógicos

- **Aritméticos:** `+`, `-`, `*`, `/`, `%`. Operan sobre operandos numéricos (`int`, `float`). El analizador semántico verifica la compatibilidad de tipos y resuelve el tipo resultante.
- **Relacionales:** `==`, `!=`, `<`, `<=`, `>`, `>=`. Comparan dos expresiones numéricas o compatibles y retornan invariablemente un valor de tipo `bool`.
- **Lógicos:** `&&` (AND), `||` (OR), `!` (NOT unario). Requieren estrictamente operandos de tipo `bool` y retornan un resultado `bool`. El signo unario `-` aplica sobre `int` o `float`.

D. Jerarquía y Precedencia de Operadores

El analizador sintáctico (Parser) elimina la ambigüedad asociativa mediante un esquema de reglas en cascada (de menor a mayor prioridad):

1. `||` (Asociatividad Izquierda a Derecha) → Regla EBNF: `logicalOrExpr`
2. `&&` (Asociatividad Izquierda a Derecha) → Regla EBNF: `logicalAndExpr`
3. `==`, `!=` (Asociatividad Izquierda a Derecha) → Regla EBNF: `equalityExpr`
4. `<`, `<=`, `>`, `>=` (Asociatividad Izquierda a Derecha) → Regla EBNF: `relationalExpr`
5. `+`, `-` (Asociatividad Izquierda a Derecha) → Regla EBNF: `additiveExpr`
6. `*`, `/`, `%` (Asociatividad Izquierda a Derecha) → Regla EBNF: `multiplicativeExpr`
7. **Unarios:** `!`, `-` (Asociatividad Derecha a Izquierda) → Regla EBNF: `unaryExpr`

8. **Primarios: Literales, Identificadores, (), llamadas a método, a[i] →**

Regla EBNF: `primaryExpr`

14. REGLAS FORMALES DE TRANSICIÓN DE ESTADOS (ESTRUCTURAS DE CONTROL)

Descripción del Modelo de Transición

Las siguientes reglas describen la semántica de transición de estados finitos ($Q1 \rightarrow Q2$) para la ejecución y evaluación sintáctico-semántica de las estructuras de control del compilador PyScript.

Convenciones de Notación:

- **Q1:** Estado inicial de la estructura de control.
- **Q2:** Estado final o de salida al cual debe converger la transición.
- **CONDICION:** Expresión lógica o relacional evaluada en la cabecera.
- **[V] / [F]:** Evaluación verdadera (V) o falsa (F) de la condición respectiva.
- **^:** Operador de conjunción lógica (AND).
- **(S1), (S2):** Sentencias o bloques de instrucciones asociadas a cada rama.
- **(INIT), (UPDT):** Instrucciones de inicialización y actualización en las cabeceras de ciclos.

1. Condicional Simple (**if**)

Sintaxis EBNF Base:

`ifStatement ::= "if" "(" expression ")" block ;`

- **Rama Verdadero ([V]):** Si la condición se evalúa como verdadera ([V]), se ejecuta el bloque de sentencias (S1) e inmediatamente el flujo transiciona al estado final Q2.
 $Q1 \rightarrow \text{CONDICION } [V] \rightarrow (S1) \rightarrow Q2$
- **Rama Falso ([F]):** Si la condición se evalúa como falsa ([F]), se omite el bloque interno y el control pasa directamente al estado final Q2 sin ejecutar sentencias.
 $Q1 \rightarrow \text{CONDICION } [F] \rightarrow Q2$

- **Condición Compuesta Conjuntiva ([V] ^ [V]):** Para expresiones compuestas evaluadas con el operador lógico AND (&&), ambas subcondiciones deben resolver obligatoriamente en [V] para ejecutar (S1) y transicionar hacia Q2.

$$Q1 \rightarrow \text{CONDICION1 [V] } \wedge \text{CONDICION2 [V] } \rightarrow (S1) \rightarrow Q2$$

2. Condicional Doble (if-else)

Sintaxis EBNF Base:

```
ifStatement ::= "if" "(" expression ")" block "else" (
ifStatement | block ) ;
```

- **Rama Verdadero ([V]):** Si la condición evalúa a verdadero ([V]), se ejecuta la rama principal (S1) (**thenBranch**) y el flujo converge en el estado final Q2.

$$Q1 \rightarrow \text{CONDICION [V] } \rightarrow (S1) \rightarrow Q2$$
- **Rama Falso ([F]):** Si la condición evalúa a falso ([F]), el flujo se desvía a la rama alternativa (S2) (**elseBranch**) para culminar en el estado final Q2.

$$Q1 \rightarrow \text{CONDICION [F] } \rightarrow (S2) \rightarrow Q2$$
- **Evaluación Anidada Conjuntiva ([V] ^ [V]):** En el caso de condiciones compuestas por intersección lógica, si ambas resultan verdaderas se ejecuta (S1) hasta Q2; si alguna falla ([F]), se ejecuta (S2) alcanzando Q2.

$$Q1 \rightarrow \text{CONDICION1 [V] } \wedge \text{CONDICION2 [V] } \rightarrow (S1) \rightarrow Q2$$

$$Q1 \rightarrow \text{CONDICION1 [F] } \rightarrow (S2) \rightarrow Q2$$

3. Bucle Indeterminado (while)

Sintaxis EBNF Base:

```
whileStatement ::= "while" "(" expression ")" block ;
```

- **Iteración del Ciclo ([V]):** Mientras la condición lógica evalúe a [V], se ejecuta el cuerpo del ciclo (S1) y el estado retroalimenta el flujo para reevaluar nuevamente desde la entrada hasta que eventualmente la condición resulte falsa y culmine en Q2.

$$Q1 \rightarrow \text{CONDICION [V] } \rightarrow (S1) \rightarrow Q1 \rightarrow \text{CONDICION [F] } \rightarrow Q2$$
- **Salida Inmediata ([F]):** Si la condición inicial evalúa a [F] desde la primera comprobación, el cuerpo (S1) jamás se ejecuta y el control

transiciona directamente de Q1 a Q2.

$Q1 \rightarrow \text{CONDICION [F]} \rightarrow Q2$

- **Iteración con Conjunción Lógica ([V] ^ [V]):** Muestra el camino repetitivo al satisfacer múltiples cláusulas binarias mediante AND (&&) antes de que el fallo de cualquiera rompa el bucle hacia Q2.
 $Q1 \rightarrow \text{CONDICION1 [V]} \wedge \text{CONDICION2 [V]} \rightarrow (S1) \rightarrow Q1 \rightarrow \text{CONDICION1 [F]} \rightarrow Q2$

4. Bucle Determinado (for)

Sintaxis EBNF Base:

```
forStatement ::= "for" "(" forInit ";" expression ";"  
forUpdate ")" block ;
```

- **Ejecución del Ciclo ([V]):** El estado inicial Q1 ejecuta la sentencia de inicialización (INIT). Si la condición evalúa a verdadero ([V]), se ejecuta el cuerpo del ciclo (S1), seguido de la instrucción de actualización (UPDT). La secuencia se repite cíclicamente hasta que la condición evalúe a falso ([F]) y el control transicione al estado final Q2.
 $Q1 \rightarrow (\text{INIT}) \rightarrow \text{CONDICION [V]} \rightarrow (S1) \rightarrow (\text{UPDT}) \rightarrow Q1 \rightarrow \text{CONDICION [F]} \rightarrow Q2$
- **Salida Inmediata del Ciclo ([F]):** Se ejecuta la instrucción de inicialización (INIT); al evaluarse la condición como falsa ([F]) en el primer intento, se omite el cuerpo (S1) y la actualización (UPDT), transicionando directamente hacia Q2.
 $Q1 \rightarrow (\text{INIT}) \rightarrow \text{CONDICION [F]} \rightarrow Q2$
- **Ejecución Compleja con Operador AND ([V] ^ [V]):** Incorpora la evaluación conjuntiva múltiple en la cabecera del ciclo **for**. Se ejecuta (INIT), se comprueban ambas condiciones simultáneamente y, si ambas son satisfactorias, se ejecuta (S1) y la actualización (UPDT) en bucle iterativo hacia Q2.
 $Q1 \rightarrow (\text{INIT}) \rightarrow \text{CONDICION1 [V]} \wedge \text{CONDICION2 [V]} \rightarrow (S1) \rightarrow (\text{UPDT}) \rightarrow Q1 \rightarrow \text{CONDICION1 [F]} \rightarrow Q2$